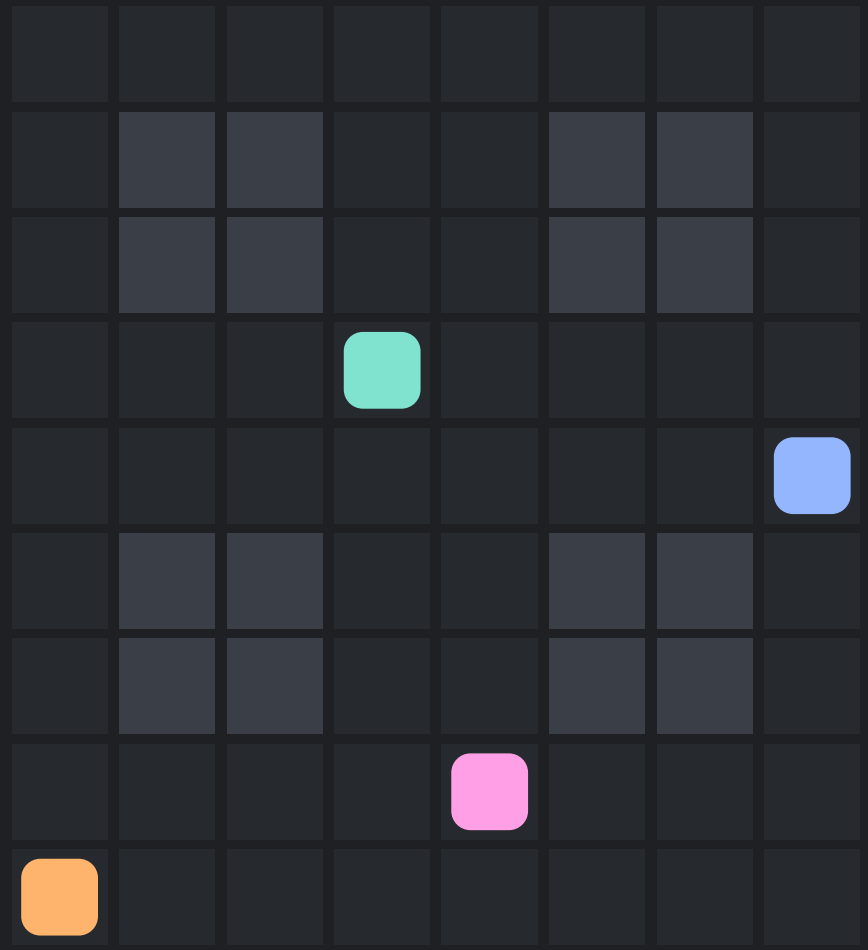
INŻYNIERIA OPROGRAMOWANIA · CHECKPOINT 1

RoboFleet

Wymagania, przypadki użycia i diagramy klas symulatora robotów magazynowych



Anna Granos · Mikołaj Zawrotniak · Patrick Fongang · Bartek Dźwigała

Wydział MiNI, Politechnika Warszawska · październik 2026

IDEA

Wirtualny magazyn, w którym roboty dzielą wąskie alejki

1

Administrator rysuje magazyn

Siatka 20 × 20 z regałami, punktami odbioru i dostawy oraz polem startowym każdego robota

2

Klient dodaje zlecenia

Przewieź towar z punktu A do punktu B. Tyle zleceń, ile potrzeba

3

Backend steruje flotą

Wybiera robota do każdego zlecenia, planuje trasy bez kolizji i pokazuje ruch na żywo

Roboty są tylko symulowane: bez sprzętu, bez uczenia maszynowego, bez płatnych usług.

AKTORZY

Kto korzysta z systemu



Gość

Rejestruje się i loguje



Klient

Tworzy przebiegi i zlecenia, uruchamia je, analizuje wyniki



Administrator

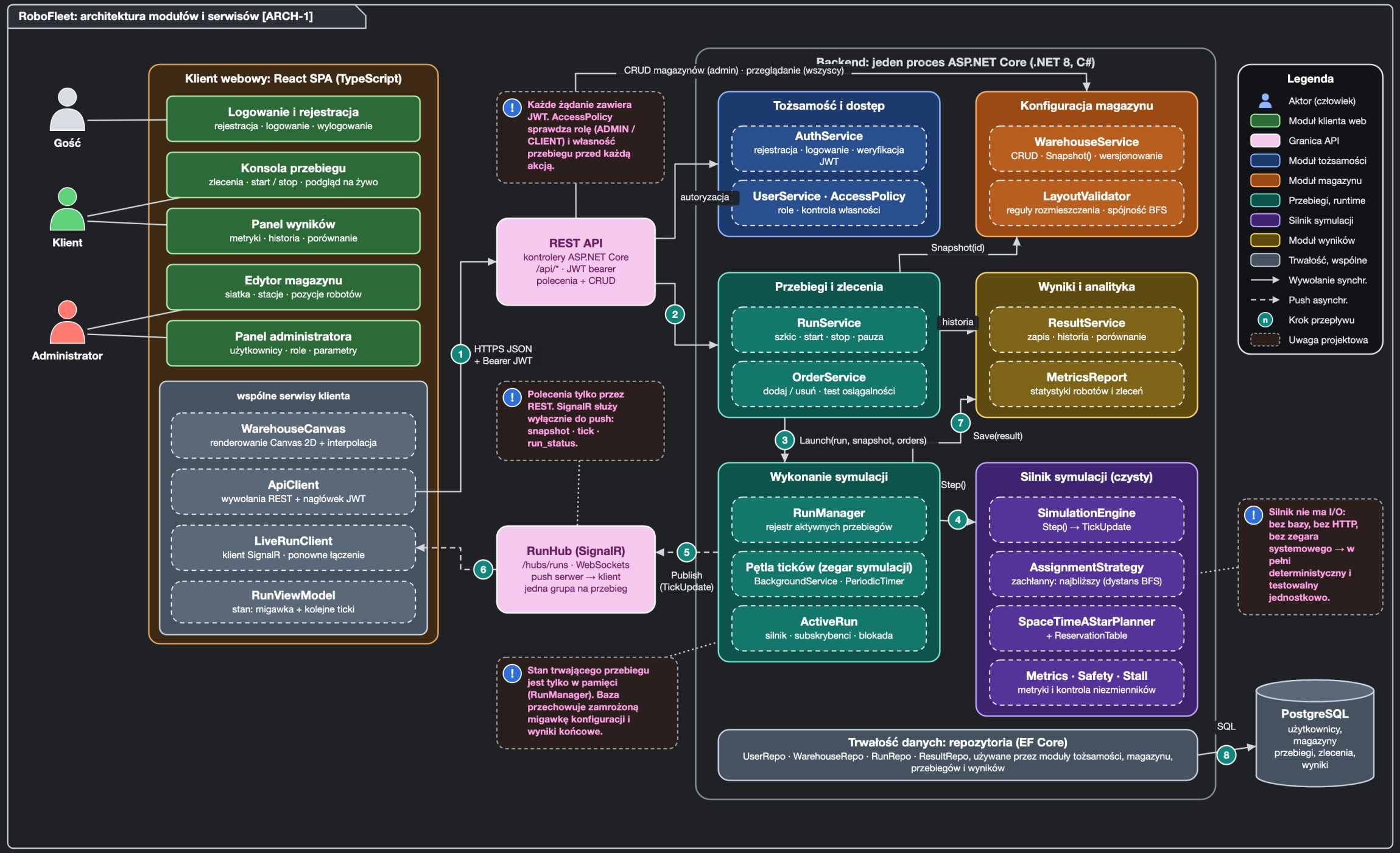
Zarządza użytkownikami i projektuje magazyny



Zegar

Zegar symulacji. Nie człowiek, tylko timer wyzwalający każdy tick

Klient i Administrator są Zarejestrowanymi użytkownikami: przeglądanie magazynów, podgląd przebiegów i wyniki działają u nich tak samo.


RoboFleet: architektura modułów i serwisów [ARCH-1]
Gość
Klient
Administrator
Klient webowy: React SPA (TypeScript)
Logowanie i rejestracja
rejestracja · logowanie · wylogowanie
Konsola przebiegu
zlecenia · start / stop · podgląd na żywo
Panel wyników
metryki · historia · porównanie
Edytor magazynu
siatka · stacje · pozycje robotów
Panel administratora
użytkownicy · role · parametry
wspólne serwisy klienta
WarehouseCanvas
renderowanie Canvas 2D + interpolacja
ApiClient
wywołania REST + nagłówek JWT
LiveRunClient
klient SignalR · ponowne łączenie
RunViewModel
stan: migawka + kolejne ticki
1
HTTPS JSON
+ Bearer JWT
Każde żądanie zawiera JWT. AccessPolicy sprawdza rolę (ADMIN / CLIENT) i własność przebiegu przed każdą akcją.
Polecenia tylko przez REST. SignalR służy wyłącznie do push: snapshot · tick · run_status.
Stan trwającego przebiegu jest tylko w pamięci (RunManager). Baza przechowuje zamrożoną migawkę konfiguracji i wyniki końcowe.
REST API
kontrolery ASP.NET Core
/api/* · JWT bearer
polecenia + CRUD
RunHub (SignalR)
/hubs/runs · WebSockets
push serwer → klient
jedna grupa na przebieg
6
5
Publish
(TickUpdate)
CRUD magazynów (admin) · przeglądanie (wszyscy)
autoryzacja
2
Backend: jeden proces ASP.NET Core (.NET 8, C#)
Tożsamość i dostęp
AuthService
rejestracja · logowanie · weryfikacja JWT
UserService · AccessPolicy
role · kontrola własności
Konfiguracja magazynu
WarehouseService
CRUD · Snapshot() · wersjonowanie
LayoutValidator
reguły rozmieszczenia · spójność BFS
Snapshot(id)
Przebiegi i zlecenia
RunService
szkic · start · stop · pauza
OrderService
dodaj / usuń · test osiągalności
historia
Wyniki i analityka
ResultService
zapis · historia · porównanie
MetricsReport
statystyki robotów i zleceń
3
Launch(run, snapshot, orders)
7
Save(result)
Wykonanie symulacji
RunManager
rejestr aktywnych przebiegów
Pętla ticków (zegar symulacji)
BackgroundService · PeriodicTimer
ActiveRun
silnik · subskrybenci · blokada
Step()
4
Silnik symulacji (czysty)
SimulationEngine
Step() → TickUpdate
AssignmentStrategy
zachłanny: najbliższy (dystans BFS)
SpaceTimeAStarPlanner
+ ReservationTable
Metrics · Safety · Stall
metryki i kontrola niezmienników
Silnik nie ma I/O: bez bazy, bez HTTP, bez zegara systemowego → w pełni deterministyczny i testowalny jednostkowo.
Trwałość danych: repozytoria (EF Core)
UserRepo · WarehouseRepo · RunRepo · ResultRepo, używane przez moduły tożsamości, magazynu, przebiegów i wyników
SQL
8
PostgreSQL
użytkownicy, magazyny przebiegi, zlecenia, wyniki
Legenda
Aktor (człowiek)
Moduł klienta web
Granica API
Moduł tożsamości
Moduł magazynu
Przebiegi, runtime
Silnik symulacji
Moduł wyników
Trwałość, wspólne
Wywołanie synchr.
Push asynchr.
Krok przepływu
Uwaga projektowa

WYMAGANIA

29 historyjek użytkownika, priorytety MoSCoW

19 Must

5 Should

5 Could

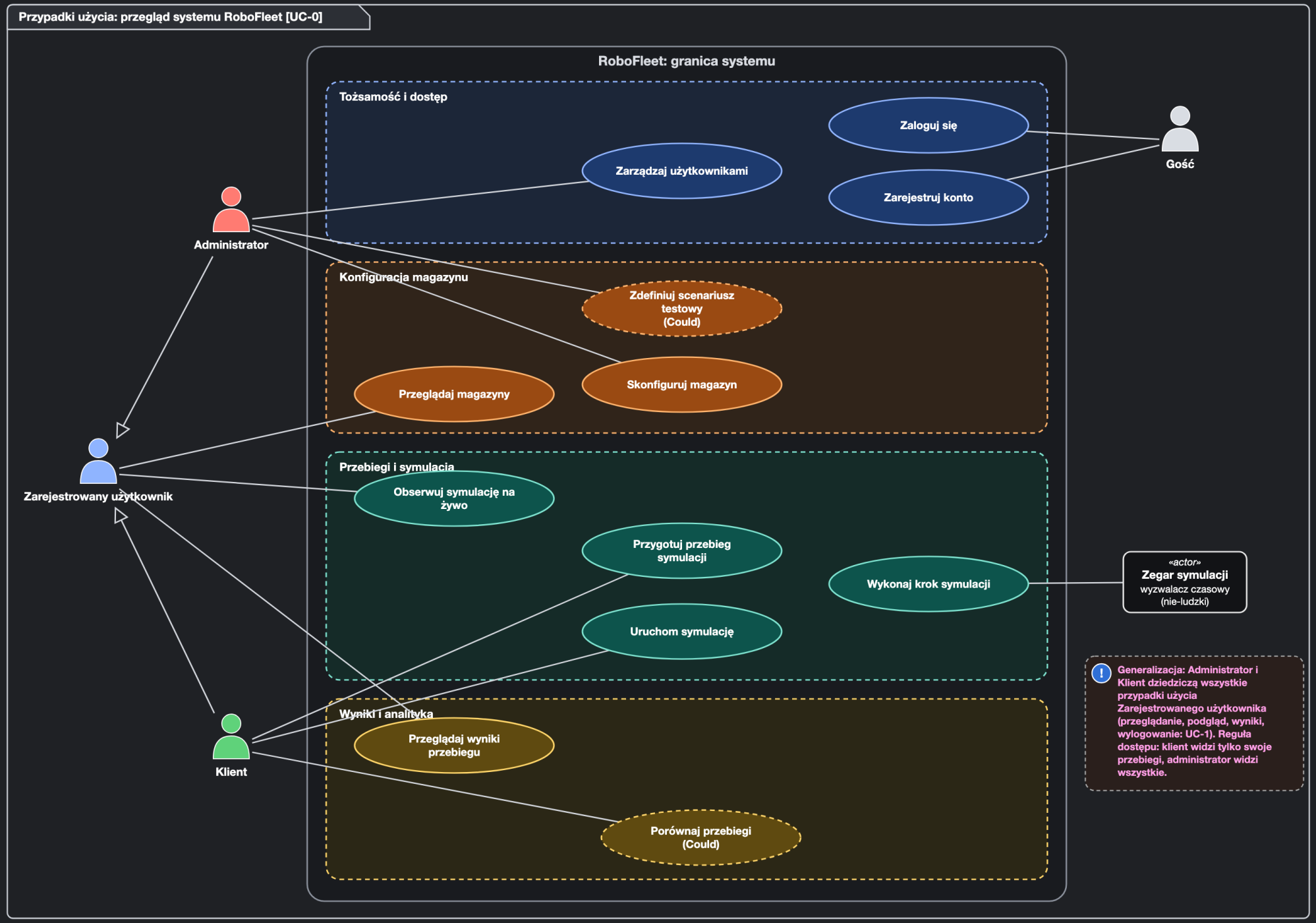
03.04 · Klient · Must

Jako klient chcę uruchomić symulację z automatycznym przydziałem zleceń i planowaniem tras, aby nie planować pracy robotów ręcznie.

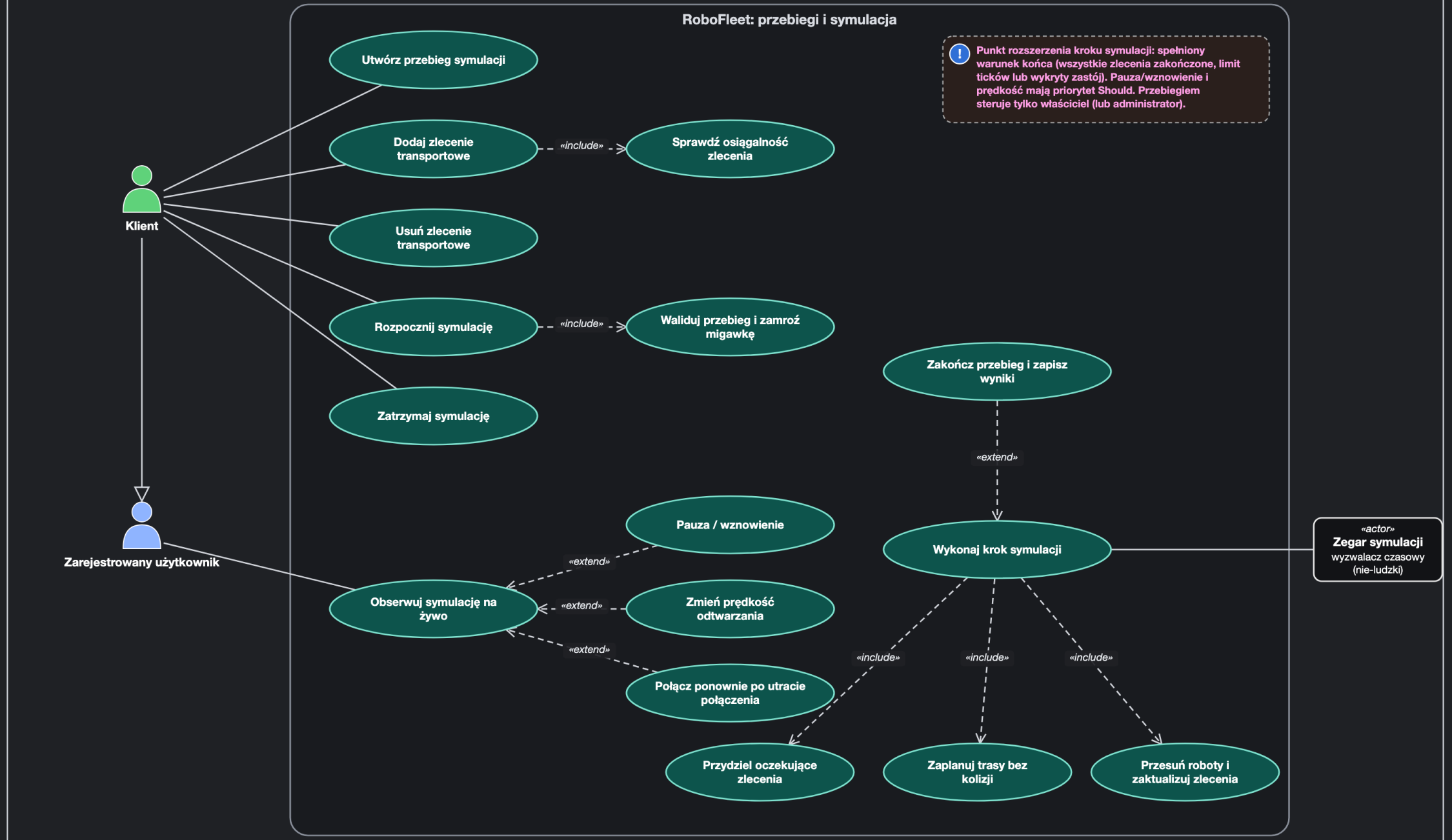
02.05 · Administrator · Must

Jako administrator chcę otrzymać wynik walidacji przed zapisem układu, aby błędne układy nie trafiały do klientów.

Poza zakresem: prawdziwe roboty, ML, baterie i ładowanie, widok 3D.


Przypadki użycia: przegląd systemu RoboFleet [UC-0]
RoboFleet: granica systemu
Tożsamość i dostęp
Zaloguj się
Zarządzaj użytkownikami
Zarejestruj konto
Gość
Administrator
Konfiguracja magazynu
Zdefiniuj scenariusz testowy (Could)
Przeglądaj magazyny
Skonfiguruj magazyn
Zarejestrowany użytkownik
Przebiegi i symulacja
Obserwuj symulację na żywo
Przygotuj przebieg symulacji
Wykonaj krok symulacji
«actor» Zegar symulacji wyzwalacz czasowy (nie-ludzki)
Uruchom symulację
Wyniki i analityka
Przeglądaj wyniki przebiegu
Klient
Porównaj przebiegi (Could)
Generalizacja: Administrator i Klient dziedziczą wszystkie przypadki użycia Zarejestrowanego użytkownika (przeglądanie, podgląd, wyniki, wylogowanie: UC-1). Reguła dostępu: klient widzi tylko swoje przebiegi, administrator widzi wszystkie.

Przebiegi i symulacja: przypadki użycia [UC-3]



KARTA HISTORYJKI

Karta historyjki 03.04: automatyczny przydział i trasy

Jako klient chcę uruchomić symulację z automatycznym przydziałem zleceń i planowaniem tras, aby nie planować pracy robotów ręcznie.

Kryteria akceptacji

- ✓ Start tylko z co najmniej jednym zleceniem?
- ✓ Zlecenie trafia do najbliższego wolnego robota (długość ścieżki)?
- ✓ Kolejka FIFO, gdy wszystkie roboty są zajęte?
- ✓ Robot czeka lub planuje ponownie zamiast wjechać na zajęte pole?
- ✓ Przebieg kończy się sam, gdy zlecenia są zakończone?

Niefunkcjonalne

- ! Zawsze 0 kolizji (wspólne pole, zamiana miejsc)
- ! Te same dane dają ten sam wynik
- ! Tick dla 10 robotów poniżej 100 ms
- ! Sterować przebiegiem może tylko właściciel lub admin

OPIS WG COCKBURNA

Przypadek użycia: wykonaj krok symulacji

Aktor

Zegar symulacji

Wyzwalacz

Upływa interwał timera

Warunek wstępny

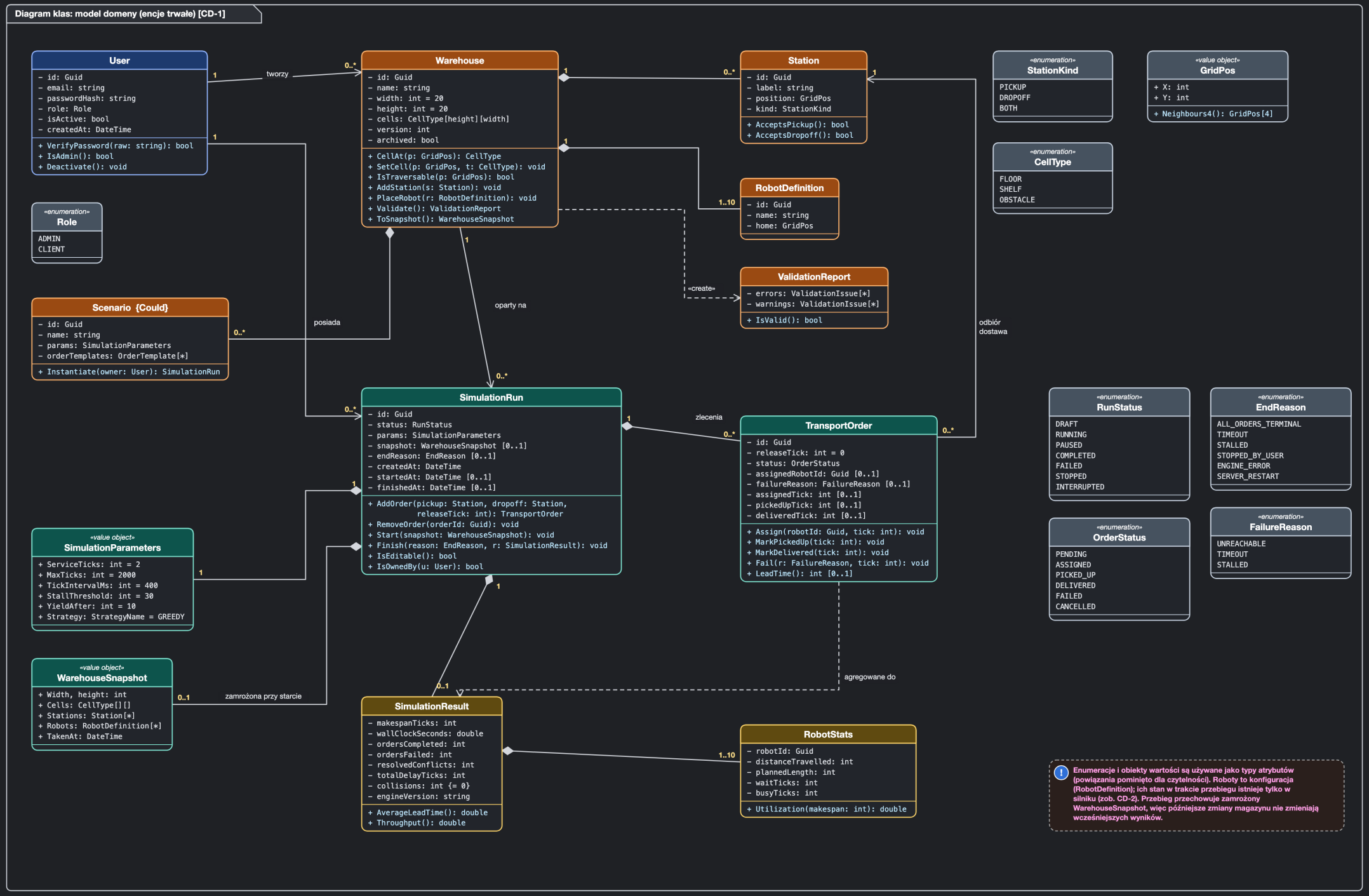
Przebieg RUNNING

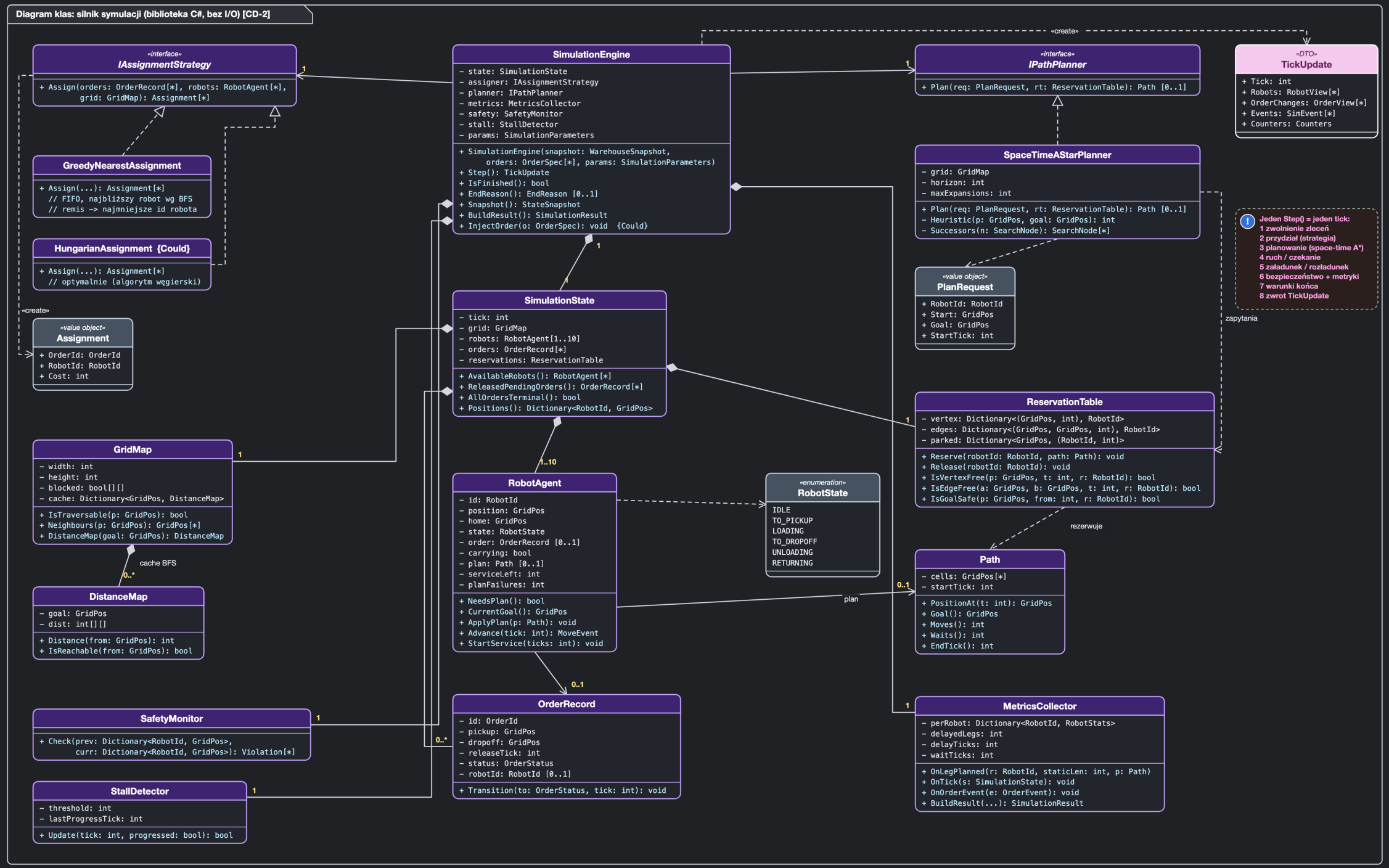
Warunek końcowy

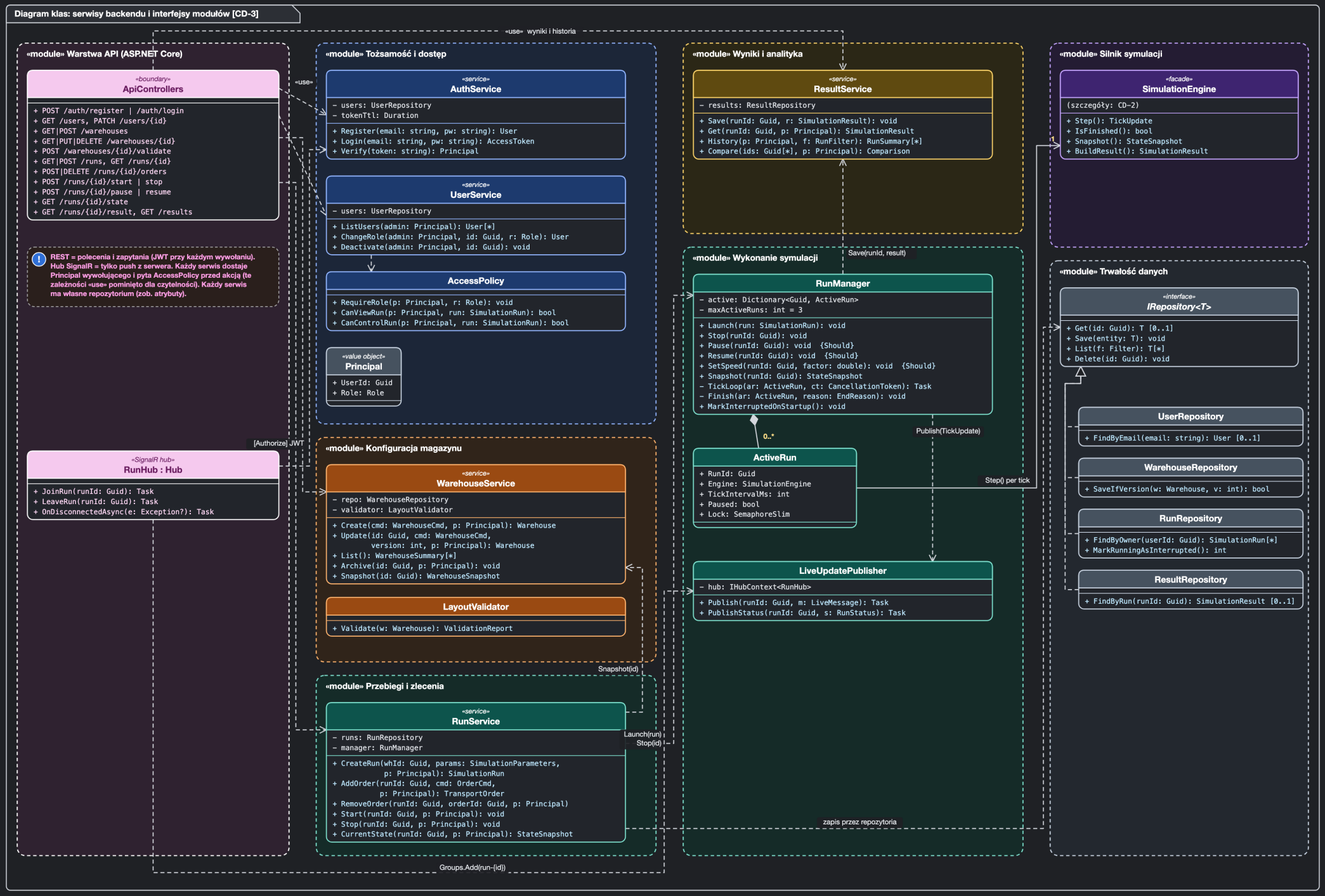
Brak wspólnych pól i zamian, aktualizacja wysłana

1. Zwolnij nowe zlecenia
2. Przydziel najbliższemu robotowi
3. Zaplanuj trasy (space-time A*)
4. Ruch lub czekanie
5. Załadunek / rozładunek
6. Bezpieczeństwo i statystyki
7. Sprawdź koniec przebiegu
8. Wyślij aktualizację

Gdy coś idzie źle: robot bez trasy czeka i próbuje ponownie. Brak postępu przez K ticków lub przekroczony limit ticków kończy przebieg jako FAILED z przyczyną.


Diagram klas: model domeny (encje trwałe) [CD-1]
User
- id: Guid
- email: string
- passwordHash: string
- role: Role
- isActive: bool
- createdAt: DateTime
+ VerifyPassword(raw: string): bool
+ IsAdmin(): bool
+ Deactivate(): void
tworzy
1
0..*
Warehouse
- id: Guid
- name: string
- width: int = 20
- height: int = 20
- cells: CellType[height][width]
- version: int
- archived: bool
+ CellAt(p: GridPos): CellType
+ SetCell(p: GridPos, t: CellType): void
+ IsTraversable(p: GridPos): bool
+ AddStation(s: Station): void
+ PlaceRobot(r: RobotDefinition): void
+ Validate(): ValidationReport
+ ToSnapshot(): WarehouseSnapshot
Station
- id: Guid
- label: string
- position: GridPos
- kind: StationKind
+ AcceptsPickup(): bool
+ AcceptsDropoff(): bool
«enumeration»
StationKind
PICKUP
DROPOFF
BOTH
«value object»
GridPos
+ X: int
+ Y: int
+ Neighbours4(): GridPos[4]
«enumeration»
Role
ADMIN
CLIENT
RobotDefinition
- id: Guid
- name: string
- home: GridPos
1..10
«enumeration»
CellType
FLOOR
SHELF
OBSTACLE
«create»
ValidationReport
- errors: ValidationIssue[*]
- warnings: ValidationIssue[*]
+ IsValid(): bool
Scenario {Could}
- id: Guid
- name: string
- params: SimulationParameters
- orderTemplates: OrderTemplate[*]
+ Instantiate(owner: User): SimulationRun
posiada
oparty na
odbiór
dostawa
SimulationRun
- id: Guid
- status: RunStatus
- params: SimulationParameters
- snapshot: WarehouseSnapshot [0..1]
- endReason: EndReason [0..1]
- createdAt: DateTime
- startedAt: DateTime [0..1]
- finishedAt: DateTime [0..1]
+ AddOrder(pickup: Station, dropoff: Station,
releaseTick: int): TransportOrder
+ RemoveOrder(orderId: Guid): void
+ Start(snapshot: WarehouseSnapshot): void
+ Finish(reason: EndReason, r: SimulationResult): void
+ IsEditable(): bool
+ IsOwnedBy(u: User): bool
zlecenia
TransportOrder
- id: Guid
- releaseTick: int = 0
- status: OrderStatus
- assignedRobotId: Guid [0..1]
- failureReason: FailureReason [0..1]
- assignedTick: int [0..1]
- pickedUpTick: int [0..1]
- deliveredTick: int [0..1]
+ Assign(robotId: Guid, tick: int): void
+ MarkPickedUp(tick: int): void
+ MarkDelivered(tick: int): void
+ Fail(r: FailureReason, tick: int): void
+ LeadTime(): int [0..1]
«enumeration»
RunStatus
DRAFT
RUNNING
PAUSED
COMPLETED
FAILED
STOPPED
INTERRUPTED
«enumeration»
EndReason
ALL_ORDERS_TERMINAL
TIMEOUT
STALLED
STOPPED_BY_USER
ENGINE_ERROR
SERVER_RESTART
«enumeration»
OrderStatus
PENDING
ASSIGNED
PICKED_UP
DELIVERED
FAILED
CANCELLED
«enumeration»
FailureReason
UNREACHABLE
TIMEOUT
STALLED
«value object»
SimulationParameters
+ ServiceTicks: int = 2
+ MaxTicks: int = 2000
+ TickIntervalMs: int = 400
+ StallThreshold: int = 30
+ YieldAfter: int = 10
+ Strategy: StrategyName = GREEDY
«value object»
WarehouseSnapshot
+ Width, height: int
+ Cells: CellType[][]
+ Stations: Station[*]
+ Robots: RobotDefinition[*]
+ TakenAt: DateTime
zamrożona przy starcie
agregowane do
SimulationResult
- makespanTicks: int
- wallClockSeconds: double
- ordersCompleted: int
- ordersFailed: int
- resolvedConflicts: int
- totalDelayTicks: int
- collisions: int {= 0}
- engineVersion: string
+ AverageLeadTime(): double
+ Throughput(): double
RobotStats
- robotId: Guid
- distanceTravelled: int
- plannedLength: int
- waitTicks: int
- busyTicks: int
+ Utilization(makespan: int): double
Enumeracje i obiekty wartości są używane jako typy atrybutów (powiązania pominięto dla czytelności). Roboty to konfiguracja (RobotDefinition); ich stan w trakcie przebiegu istnieje tylko w silniku (zob. CD-2). Przebieg przechowuje zamrożony WarehouseSnapshot, więc późniejsze zmiany magazynu nie zmieniają wcześniejszych wyników.


Diagram klas: silnik symulacji (biblioteka C#, bez I/O) [CD-2]
«create»
«interface»
IAssignmentStrategy
+ Assign(orders: OrderRecord[*], robots: RobotAgent[*], grid: GridMap): Assignment[*]
SimulationEngine
- state: SimulationState
- assigner: IAssignmentStrategy
- planner: IPathPlanner
- metrics: MetricsCollector
- safety: SafetyMonitor
- stall: StallDetector
- params: SimulationParameters
+ SimulationEngine(snapshot: WarehouseSnapshot, orders: OrderSpec[*], params: SimulationParameters)
+ Step(): TickUpdate
+ IsFinished(): bool
+ EndReason(): EndReason [0..1]
+ Snapshot(): StateSnapshot
+ BuildResult(): SimulationResult
+ InjectOrder(o: OrderSpec): void {Could}
«interface»
IPathPlanner
+ Plan(req: PlanRequest, rt: ReservationTable): Path [0..1]
«DTO»
TickUpdate
+ Tick: int
+ Robots: RobotView[*]
+ OrderChanges: OrderView[*]
+ Events: SimEvent[*]
+ Counters: Counters
GreedyNearestAssignment
+ Assign(...): Assignment[*]
// FIFO, najbliższy robot wg BFS
// remis -> najmniejsze id robota
SpaceTimeAStarPlanner
- grid: GridMap
- horizon: int
- maxExpansions: int
+ Plan(req: PlanRequest, rt: ReservationTable): Path [0..1]
- Heuristic(p: GridPos, goal: GridPos): int
- Successors(n: SearchNode): SearchNode[*]
HungarianAssignment {Could}
+ Assign(...): Assignment[*]
// optymalnie (algorytm węgierski)
Jeden Step() = jeden tick:
1 zwolnienie zleceń
2 przydział (strategia)
3 planowanie (space-time A*)
4 ruch / czekanie
5 załadunek / rozładunek
6 bezpieczeństwo + metryki
7 warunki końca
8 zwrot TickUpdate
zapytania
«create»
«value object»
Assignment
+ OrderId: OrderId
+ RobotId: RobotId
+ Cost: int
«value object»
PlanRequest
+ RobotId: RobotId
+ Start: GridPos
+ Goal: GridPos
+ StartTick: int
SimulationState
- tick: int
- grid: GridMap
- robots: RobotAgent[1..10]
- orders: OrderRecord[*]
- reservations: ReservationTable
+ AvailableRobots(): RobotAgent[*]
+ ReleasedPendingOrders(): OrderRecord[*]
+ AllOrdersTerminal(): bool
+ Positions(): Dictionary<RobotId, GridPos>
ReservationTable
- vertex: Dictionary<(GridPos, int), RobotId>
- edges: Dictionary<(GridPos, GridPos, int), RobotId>
- parked: Dictionary<GridPos, (RobotId, int)>
+ Reserve(robotId: RobotId, path: Path): void
+ Release(robotId: RobotId): void
+ IsVertexFree(p: GridPos, t: int, r: RobotId): bool
+ IsEdgeFree(a: GridPos, b: GridPos, t: int, r: RobotId): bool
+ IsGoalSafe(p: GridPos, from: int, r: RobotId): bool
GridMap
- width: int
- height: int
- blocked: bool[][]
- cache: Dictionary<GridPos, DistanceMap>
+ IsTraversable(p: GridPos): bool
+ Neighbours(p: GridPos): GridPos[*]
+ DistanceMap(goal: GridPos): DistanceMap
RobotAgent
- id: RobotId
- position: GridPos
- home: GridPos
- state: RobotState
- order: OrderRecord [0..1]
- carrying: bool
- plan: Path [0..1]
- serviceLeft: int
- planFailures: int
+ NeedsPlan(): bool
+ CurrentGoal(): GridPos
+ ApplyPlan(p: Path): void
+ Advance(tick: int): MoveEvent
+ StartService(ticks: int): void
«enumeration»
RobotState
IDLE
TO_PICKUP
LOADING
TO_DROPOFF
UNLOADING
RETURNING
rezerwuje
cache BFS
Path
- cells: GridPos[*]
- startTick: int
+ PositionAt(t: int): GridPos
+ Goal(): GridPos
+ Moves(): int
+ Waits(): int
+ EndTick(): int
plan
DistanceMap
- goal: GridPos
- dist: int[][]
+ Distance(from: GridPos): int
+ IsReachable(from: GridPos): bool
OrderRecord
- id: OrderId
- pickup: GridPos
- dropoff: GridPos
- releaseTick: int
- status: OrderStatus
- robotId: RobotId [0..1]
+ Transition(to: OrderStatus, tick: int): void
SafetyMonitor
+ Check(prev: Dictionary<RobotId, GridPos>, curr: Dictionary<RobotId, GridPos>): Violation[*]
MetricsCollector
- perRobot: Dictionary<RobotId, RobotStats>
- delayedLegs: int
- delayTicks: int
- waitTicks: int
+ OnLegPlanned(r: RobotId, staticLen: int, p: Path)
+ OnTick(s: SimulationState): void
+ OnOrderEvent(e: OrderEvent): void
+ BuildResult(...): SimulationResult
StallDetector
- threshold: int
- lastProgressTick: int
+ Update(tick: int, progressed: bool): bool
1
0..*
1..10
0..1


Diagram klas: serwisy backendu i interfejsy modułów [CD-3]
«use» wyniki i historia
«module» Warstwa API (ASP.NET Core)
«boundary»
ApiControllers
+ POST /auth/register | /auth/login
+ GET /users, PATCH /users/{id}
+ GET|POST /warehouses
+ GET|PUT|DELETE /warehouses/{id}
+ POST /warehouses/{id}/validate
+ GET|POST /runs, GET /runs/{id}
+ POST|DELETE /runs/{id}/orders
+ POST /runs/{id}/start | stop
+ POST /runs/{id}/pause | resume
+ GET /runs/{id}/state
+ GET /runs/{id}/result, GET /results
«use»
1
REST = polecenia i zapytania (JWT przy każdym wywołaniu). Hub SignalR = tylko push z serwera. Każdy serwis dostaje Principal wywołującego i pyta AccessPolicy przed akcją (te zależności «use» pominięto dla czytelności). Każdy serwis ma własne repozytorium (zob. atrybuty).
«SignalR hub»
RunHub : Hub
+ JoinRun(runId: Guid): Task
+ LeaveRun(runId: Guid): Task
+ OnDisconnectedAsync(e: Exception?): Task
[Authorize] JWT
«module» Tożsamość i dostęp
«service»
AuthService
- users: UserRepository
- tokenTtl: Duration
+ Register(email: string, pw: string): User
+ Login(email: string, pw: string): AccessToken
+ Verify(token: string): Principal
«service»
UserService
- users: UserRepository
+ ListUsers(admin: Principal): User[*]
+ ChangeRole(admin: Principal, id: Guid, r: Role): User
+ Deactivate(admin: Principal, id: Guid): void
AccessPolicy
+ RequireRole(p: Principal, r: Role): void
+ CanViewRun(p: Principal, run: SimulationRun): bool
+ CanControlRun(p: Principal, run: SimulationRun): bool
«value object»
Principal
+ UserId: Guid
+ Role: Role
«module» Konfiguracja magazynu
«service»
WarehouseService
- repo: WarehouseRepository
- validator: LayoutValidator
+ Create(cmd: WarehouseCmd, p: Principal): Warehouse
+ Update(id: Guid, cmd: WarehouseCmd, version: int, p: Principal): Warehouse
+ List(): WarehouseSummary[*]
+ Archive(id: Guid, p: Principal): void
+ Snapshot(id: Guid): WarehouseSnapshot
LayoutValidator
+ Validate(w: Warehouse): ValidationReport
Snapshot(id)
«module» Przebiegi i zlecenia
«service»
RunService
- runs: RunRepository
- manager: RunManager
+ CreateRun(whId: Guid, params: SimulationParameters, p: Principal): SimulationRun
+ AddOrder(runId: Guid, cmd: OrderCmd, p: Principal): TransportOrder
+ RemoveOrder(runId: Guid, orderId: Guid, p: Principal)
+ Start(runId: Guid, p: Principal): void
+ Stop(runId: Guid, p: Principal): void
+ CurrentState(runId: Guid, p: Principal): StateSnapshot
Launch(run)
Stop(id)
Groups.Add(run-{id})
«module» Wyniki i analityka
«service»
ResultService
- results: ResultRepository
+ Save(runId: Guid, r: SimulationResult): void
+ Get(runId: Guid, p: Principal): SimulationResult
+ History(p: Principal, f: RunFilter): RunSummary[*]
+ Compare(ids: Guid[*], p: Principal): Comparison
Save(runId, result)
«module» Wykonanie symulacji
RunManager
- active: Dictionary<Guid, ActiveRun>
- maxActiveRuns: int = 3
+ Launch(run: SimulationRun): void
+ Stop(runId: Guid): void
+ Pause(runId: Guid): void {Should}
+ Resume(runId: Guid): void {Should}
+ SetSpeed(runId: Guid, factor: double): void {Should}
+ Snapshot(runId: Guid): StateSnapshot
- TickLoop(ar: ActiveRun, ct: CancellationToken): Task
- Finish(ar: ActiveRun, reason: EndReason): void
+ MarkInterruptedOnStartup(): void
0..*
ActiveRun
+ RunId: Guid
+ Engine: SimulationEngine
+ TickIntervalMs: int
+ Paused: bool
+ Lock: SemaphoreSlim
Publish(TickUpdate)
Step() per tick
LiveUpdatePublisher
- hub: IHubContext<RunHub>
+ Publish(runId: Guid, m: LiveMessage): Task
+ PublishStatus(runId: Guid, s: RunStatus): Task
zapis przez repozytoria
«module» Silnik symulacji
«facade»
SimulationEngine
(szczegóły: CD-2)
+ Step(): TickUpdate
+ IsFinished(): bool
+ Snapshot(): StateSnapshot
+ BuildResult(): SimulationResult
1
«module» Trwałość danych
«interface»
IRepository<T>
+ Get(id: Guid): T [0..1]
+ Save(entity: T): void
+ List(f: Filter): T[*]
+ Delete(id: Guid): void
UserRepository
+ FindByEmail(email: string): User [0..1]
WarehouseRepository
+ SaveIfVersion(w: Warehouse, v: int): bool
RunRepository
+ FindByOwner(userId: Guid): SimulationRun[*]
+ MarkRunningAsInterrupted(): int
ResultRepository
+ FindByRun(runId: Guid): SimulationResult [0..1]

PODSUMOWANIE

Otwarte kwestie i kolejne kroki

Otwarte kwestie

Czy klient może dodawać zlecenia w trakcie przebiegu? Na razie nie.

Czy wystarczy przydział zachłanny, czy dodać optymalny do porównania?

Następny krok

Pierwszy działający fragment: jeden robot, jedno zlecenie, ruch na żywo.

Potem logowanie, edytor siatki i planowanie wielu robotów.

Dziękujemy! Pytania?